ENGINEERING INTELLIGENCE PLATFORM

Business Requirements & Scope Finalization

Document Type: Product Requirements Document (PRD)
Phase: Phase 1 — Business Requirements & Scope Finalization
Version: 1.0
Status: Draft — Pending Finalization of Access and Knowledge Organization Decisions
Product: Engineering Intelligence Platform

---

1. Executive Summary

The Engineering Intelligence Platform is an AI-powered knowledge and investigation system designed to help engineering teams search, understand, analyze, and investigate technical information stored across organizational documents.

Engineering teams regularly work with large volumes of technical information, including incident reports, system architecture documents, troubleshooting guides, deployment procedures, infrastructure reports, operational records, and postmortems.

This information is often fragmented across different files and repositories. Engineers must manually search through documents, compare information, identify relationships, and reconstruct the context behind technical problems.

The proposed platform centralizes this knowledge and makes it accessible through an intelligent conversational interface powered by Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and AI agents.

The platform will not simply generate answers from documents. It will prioritize evidence, source traceability, factual accuracy, uncertainty handling, and structured technical analysis.

The primary objective is to reduce engineering investigation time while improving the reliability and traceability of technical decisions.

2. Business Problem

2.1 Current Challenges

Engineering organizations generate and maintain substantial amounts of technical documentation throughout their operations.

However, the information contained in these documents is not always easy to access or use.

The major challenges include:

1. Fragmented Technical Knowledge

Technical information is distributed across multiple documents, including:

- Incident reports and postmortems.
- System architecture documentation.
- Infrastructure and deployment reports.
- Troubleshooting guides.
- Operational procedures.
- Technical specifications.
- Runbooks and engineering guidelines.

Engineers often need to search through multiple files to understand a single issue.

2. Time-Consuming Investigation

When an incident or technical problem occurs, engineers need to understand:

- What happened?
- When did it happen?
- What components were affected?
- What was the reported root cause?
- How was the issue resolved?
- Has a similar issue occurred before?

Finding these answers manually can consume significant engineering time.

3. Lack of Cross-Document Intelligence

Individual documents may contain only part of the information needed to understand a problem.

For example:

- An incident report describes a database connection failure.
- An architecture document explains the database connection architecture.
- A runbook describes the recommended troubleshooting procedure.

- A postmortem describes the final resolution.

Engineers need to connect information across these documents to understand the complete situation.

4. Inconsistent or Conflicting Information

Different documents may contain outdated, incomplete, or conflicting technical information.

An AI system must not blindly combine conflicting statements or present assumptions as facts.

5. Limited Traceability of AI-Generated Answers

A conventional AI chatbot may generate technically convincing answers without showing where the information came from.

In engineering environments, unsupported answers can lead to incorrect investigations and poor technical decisions.

The platform must therefore provide evidence-backed responses with references to the original documents.

2.2 Business Opportunity

The opportunity is to transform static technical documentation into an interactive engineering knowledge system.

Instead of manually searching through documents, engineers should be able to ask natural-language questions and receive structured responses supported by relevant evidence.

The platform should help engineers move from document discovery to technical understanding and investigation.

3. Product Vision

To build a reliable AI-powered engineering knowledge platform that enables engineering teams to discover, understand, verify, and analyze technical knowledge across organizational documents.

The platform should function as an intelligent engineering knowledge assistant rather than a generic document chatbot.

3.1 Core Product Principles

The platform will be designed around the following principles:

Evidence First: Important factual claims must be supported by retrieved source evidence.

Traceability: Users must be able to identify the document and relevant passage supporting an answer.

Honest Uncertainty: The system must communicate when information is insufficient, ambiguous, outdated, or conflicting.

Context Awareness: The system must understand follow-up questions within the context of previous conversations.

Controlled Intelligence: AI agents must operate through explicitly defined tools with validated parameters and execution limits.

Reliability: Processing failures, model failures, and tool failures must be handled without silently producing misleading results.

Security: Documents and conversations must only be accessible to authorized users.

4. Target Users and Personas

The initial product will focus on engineering and technical operations teams.

4.1 Software Engineers

Typical responsibilities:

- Investigating application errors.
- Understanding system architecture.
- Debugging software failures.
- Reviewing technical specifications.
- Finding previous solutions to similar problems.

Expected platform value:

Quickly retrieve relevant technical information and understand how previous engineering issues were investigated and resolved.

4.2 DevOps Engineers

Typical responsibilities:

- Managing deployments.
- Investigating infrastructure failures.
- Troubleshooting CI/CD pipelines.

- Understanding cloud infrastructure.
- Reviewing operational procedures.

Expected platform value:

Connect infrastructure documentation, deployment procedures, and incident records to support faster technical investigations.

4.3 Site Reliability Engineers (SREs)

Typical responsibilities:

- Investigating service outages.
- Analyzing recurring incidents.
- Reviewing reliability reports.
- Understanding system failure patterns.
- Evaluating operational risks.

Expected platform value:

Compare incidents, identify recurring failure patterns, and generate evidence-backed investigation reports.

4.4 Technical Support Engineers

Typical responsibilities:

- Investigating technical customer issues.
- Finding known failure scenarios.
- Reviewing troubleshooting procedures.
- Identifying relevant technical documentation.

Expected platform value:

Retrieve relevant technical knowledge and provide consistent, source-backed troubleshooting information.

5. Product Scope

5.1 Initial Knowledge Domain

The initial version will focus exclusively on engineering and technical operations knowledge.

Supported knowledge categories:

1. Incident reports.
2. Postmortems.
3. System architecture documents.
4. Infrastructure documentation.
5. Deployment procedures.
6. Troubleshooting guides.
7. Operational runbooks.
8. Technical specifications.

The initial demonstration dataset should contain approximately 8–12 coherent documents.

Documents may be synthetic or based on publicly available technical material. Synthetic documents must be clearly identified as demonstration data and must not be represented as real company records.

5.2 Supported File Formats

The platform must support:

- PDF
- TXT
- Markdown
- CSV

PDF will be the primary format used for the initial demonstration.

6. Functional Requirements

6.1 Document Intelligence Module

Objective: Convert uploaded technical documents into searchable and structured knowledge.

The system must:

1. Allow authorized users to upload supported documents.
2. Validate file type, size, and content.
3. Extract text and relevant document metadata.

4. Preprocess extracted content.
5. Divide documents into meaningful chunks.
6. Generate embeddings for document chunks.
7. Store vectors and associated metadata.
8. Maintain document-level processing status.
9. Detect duplicate documents or duplicate processing requests.
10. Handle extraction, embedding, and indexing failures.
11. Support retrying failed processing jobs.
12. Allow users to view and manage uploaded documents.

Expected outcome: Uploaded documents become searchable through the platform's knowledge base.

6.2 Evidence-Based Retrieval and RAG Module

Objective: Retrieve relevant technical information and generate answers grounded in organizational documents.

The system must:

1. Accept natural-language questions.
2. Convert questions into embeddings.
3. Retrieve relevant document chunks.
4. Rank retrieved information by relevance.
5. Apply appropriate context and metadata filtering.
6. Construct context for the LLM.
7. Generate answers using retrieved evidence.
8. Provide source citations.
9. Identify insufficient evidence.
10. Identify conflicting information when supported by retrieved sources.
11. Avoid presenting unsupported information as established fact.

At least one retrieval improvement must be implemented and justified.

Potential approaches include:

- Hybrid search.
- Reranking.
- Query rewriting.
- Metadata filtering.
- Contextual retrieval.

Expected outcome: Users receive relevant, verifiable answers rather than unsupported AI-generated responses.

6.3 Technical Investigation Module

Objective: Help engineers investigate technical problems using information across multiple documents.

The system should support:

1. Identifying reported causes of technical incidents.
2. Finding previous incidents with similar symptoms.
3. Comparing multiple incident reports.
4. Identifying recurring failure patterns.
5. Comparing troubleshooting procedures.
6. Identifying conflicting technical information.
7. Identifying missing information relevant to an investigation.
8. Generating evidence-backed recommendations.

An important distinction must be maintained:

- **Reported root cause:** A cause explicitly documented in a source.
- **AI-generated hypothesis:** A possible explanation inferred from available evidence.
- **Verified conclusion:** A conclusion supported by sufficient evidence.

The platform must not present an AI-generated hypothesis as a confirmed root cause.

Expected outcome: Engineers can investigate technical issues using a structured, evidence-based approach.

6.4 LLM Orchestration Module

Objective: Provide a reliable and extensible interface for interacting with multiple LLM providers.

The system must:

1. Support at least two LLM providers.
2. Initially integrate OpenAI and Anthropic.
3. Provide a common provider abstraction.
4. Support model selection.
5. Maintain reusable prompt templates.
6. Support structured output generation.
7. Validate model responses.
8. Track token consumption.
9. Handle rate limits and provider errors.
10. Implement model fallback.
11. Record relevant model execution information.

Expected outcome: The application can use multiple LLM providers without tightly coupling business logic to a single provider.

6.5 AI Agent Module

Objective: Enable controlled multi-step execution of technical knowledge tasks.

The initial agent must support at least three tools.

Tool 1: Knowledge Base Search

Purpose:

- Search technical documents.
- Retrieve relevant passages.
- Return document references and evidence.

Tool 2: Document Analysis

Purpose:

- Analyze a selected document.
- Extract important technical findings.
- Identify key events, causes, symptoms, and resolutions where documented.

Tool 3: Structured Report Generation

Purpose:

- Generate structured technical investigation reports.
- Organize findings, supporting evidence, uncertainties, and recommendations.

The agent must support:

1. Tool selection based on user requests.
2. Parameter validation.
3. Multi-step tool execution.
4. Execution limits.
5. Tool failure handling.
6. Clear execution visibility.
7. Structured final responses.

Expected outcome: Users can request multi-step technical investigations without manually executing every individual operation.

6.6 Conversational Memory Module

Objective: Maintain conversation context across multiple interactions.

The system must:

1. Support multiple conversations.
2. Persist conversation history.
3. Associate conversations with authorized users.
4. Support contextual follow-up questions.
5. Retrieve relevant previous messages.
6. Manage context length.

7. Avoid mixing information between unrelated conversations or users.

Example:

User: Why did the database connection fail?

AI: The incident report identifies connection pool exhaustion as the reported cause.

User: How was it resolved?

AI: Based on the same incident report, the team increased the connection pool limit and adjusted connection handling.

The second question should be interpreted using the previous conversation context.

Expected outcome: Users can conduct continuous technical investigations instead of starting from scratch with every question.

6.7 Dashboard and Frontend Module

Objective: Provide an accessible interface for managing technical knowledge and interacting with AI capabilities.

The frontend must include:

Authentication

- Login.
- User authentication.
- Protected application routes.
- Appropriate access restrictions.

Dashboard

- Document statistics.
- Processing status.
- Usage statistics.
- AI activity.
- Token consumption.
- Latency and error information.

Document Management

- Upload documents.
- View document lists.
- View processing status.
- View document metadata.
- Manage documents.

AI Knowledge Chat

- Ask technical questions.
- View AI responses.
- View citations.
- Access source documents.
- Continue conversations.

Conversation History

- View previous conversations.
- Open existing conversations.
- Continue previous discussions.

Agent Execution Visibility

- View tools selected by the agent.
- View execution steps.
- View tool results and failures.
- Understand how a final response was generated.

Expected outcome: Users can access the platform's core capabilities through a clear and functional interface.

6.8 Evaluation and Quality Measurement Module

Objective: Establish a repeatable method for measuring the quality and reliability of the RAG system.

The assessment requires a minimum evaluation dataset of 20 questions.

The evaluation must measure:

1. Retrieval relevance.
2. Answer correctness.
3. Source grounding.
4. Citation quality.
5. Insufficient-evidence handling.
6. Conflicting-document handling.
7. Hallucination resistance.
8. Response latency.
9. Token consumption.

Possible evaluation approaches:

- RAGAS.
- DeepEval.
- LangSmith.
- A custom evaluation framework.

The final evaluation report must explain:

- Dataset composition.
- Evaluation methodology.
- Metrics used.
- Results.
- Known limitations.
- Areas requiring improvement.

Expected outcome: The project demonstrates measurable quality rather than relying only on subjective impressions of AI responses.

7. Non-Functional Requirements

The platform must satisfy the following quality requirements.

7.1 Security

- Authentication must protect application resources.
- Users must not access unauthorized documents or conversations.
- API credentials must be stored securely.
- Sensitive configuration must use environment variables or an appropriate secret-management mechanism.
- File uploads must be validated.
- APIs must implement appropriate authorization and input validation.
- Prompt injection risks must be addressed.
- User input and retrieved document content must not be treated as trusted instructions.

7.2 Reliability

- Failed document processing must be recoverable.
- Duplicate processing jobs must be prevented.
- LLM provider failures must be handled.
- Agent execution must have limits and timeouts.
- Invalid model outputs must be detected.
- Errors must be logged appropriately.

7.3 Performance

The system should:

- Process documents without unnecessarily blocking user requests.
- Support asynchronous document processing where appropriate.
- Retrieve relevant information efficiently.
- Manage LLM context and token consumption.
- Monitor response latency.

7.4 Scalability

The architecture should allow future expansion in:

- Number of users.

- Number of documents.
- Document processing volume.
- Embedding and retrieval workloads.
- LLM providers.
- Agent tools.

7.5 Maintainability

- Maintain separation of concerns.
- Use modular backend services.
- Keep provider integrations replaceable.
- Maintain clear API contracts.
- Write automated tests.
- Document architecture and important design decisions.

7.6 Observability

The system should record:

- API errors.
- Document processing failures.
- LLM provider errors.
- Token consumption.
- Request latency.
- Retrieval execution information.
- Agent tool execution results.

8. Advanced Technical Requirements

The assessment requires implementation of at least three advanced challenges.

The proposed initial selections are:

1. Intelligent Model Fallback

When the primary LLM provider fails or becomes unavailable, the system should attempt an appropriate secondary provider.

2. Prompt Injection Protection

The platform must treat retrieved document content and user-provided text as untrusted input and prevent them from overriding system-level instructions or tool permissions.

3. Agent Reliability

The agent must implement execution limits, parameter validation, timeouts, and failure recovery.

Additional improvements may include:

- Hybrid retrieval or reranking.
- AI response verification.
- Cost optimization.
- Asynchronous document processing.

9. Primary User Workflows

Workflow 1: Document Upload and Processing

1. User logs in.
2. User opens document management.
3. User uploads a technical document.
4. Backend validates the file.
5. Document content is extracted.
6. Content is preprocessed and chunked.
7. Embeddings are generated.
8. Chunks and metadata are stored.
9. Processing status is updated.
10. Document becomes searchable.

Workflow 2: Evidence-Based Technical Question

1. User opens AI Knowledge Chat.

2. User submits a technical question.
3. Backend processes the query.
4. Retrieval service searches the knowledge base.
5. Relevant passages are ranked.
6. LLM receives the question and supporting evidence.
7. LLM generates an answer.
8. Response validation checks the output.
9. The user receives the answer with citations.
10. The user can inspect supporting source material.

Workflow 3: Technical Investigation Using the Agent

Example request:

"Investigate recurring database connection failures across our incident reports and prepare a technical report."

Expected process:

1. Agent interprets the request.
2. Agent identifies the need to search the knowledge base.
3. Knowledge Base Search retrieves relevant incidents.
4. Document Analysis examines relevant documents.
5. Agent compares the available findings.
6. Structured Report Generation creates the final report.
7. Agent returns findings, evidence, uncertainties, and recommendations.
8. User can inspect the execution history.

Workflow 4: Contextual Follow-Up

1. User asks an initial technical question.
2. Platform returns an evidence-backed answer.
3. User asks a follow-up question.

4. Platform retrieves relevant conversation context.
5. System interprets the follow-up.
6. Platform generates a context-aware response.

10. Demonstration Dataset

The initial demonstration environment should contain a coherent collection of engineering documents.

Proposed document set:

Document ID	Document Name	Purpose
DOC-001	Database Connection Failure Incident	Database failure investigation
DOC-002	API Gateway Latency Incident	API performance investigation
DOC-003	Authentication Service Failure	Authentication incident
DOC-004	System Architecture Document	System component relationships
DOC-005	Database Troubleshooting Runbook	Database troubleshooting
DOC-006	API Incident Response Runbook	API incident response
DOC-007	Deployment Procedures	Deployment workflow
DOC-008	Infrastructure Operations Guide	Infrastructure knowledge
DOC-009	Database Outage Postmortem	Root cause and resolution
DOC-010	Service Reliability Report	Recurring operational patterns

These are proposed demonstration documents, not claims about actual company records.

The documents should contain enough connected information to demonstrate:

- Single-document retrieval.
- Cross-document retrieval.
- Technical investigation.
- Follow-up questions.
- Conflicting information.

- Insufficient evidence.
- Citation traceability.

11. Acceptance Criteria

The project will be considered functionally successful when the following conditions are met.

AC-01 — Authentication

An authorized user can log in and access protected platform features.

AC-02 — Document Upload

An authorized user can upload a supported PDF document.

AC-03 — Document Processing

The uploaded document is extracted, chunked, embedded, and indexed successfully.

AC-04 — Processing Visibility

The user can view the document's processing status and identify failures.

AC-05 — Knowledge Query

The user can submit a natural-language technical question.

AC-06 — Evidence Retrieval

The system retrieves relevant supporting document passages.

AC-07 — Source Citations

The generated answer contains references that allow users to verify its supporting evidence.

AC-08 — Insufficient Evidence

The system communicates when the knowledge base does not contain enough evidence to answer a question reliably.

AC-09 — Conflicting Information

The system can identify and communicate relevant conflicts between retrieved documents.

AC-10 — Conversational Context

The system supports contextual follow-up questions within a conversation.

AC-11 — Multi-Tool Agent

The agent can use the three required tools to complete a multi-step technical task.

AC-12 — Evaluation

The project includes a repeatable evaluation dataset containing at least 20 questions and a documented evaluation report.

AC-13 — Multi-Provider LLM

The application integrates at least two LLM providers and demonstrates appropriate fallback/error handling.

AC-14 — Dashboard

The frontend exposes document management, AI chat, citations, conversation history, and agent execution visibility.

AC-15 — Cloud Deployment

The application is deployed and accessible through live frontend and backend URLs.

12. Project Scope Exclusions

To prevent unnecessary complexity, the following capabilities are explicitly excluded from the initial version.

- Merchant and distributor intelligence.
- CRM and sales automation.
- ERP functionality.
- Autonomous production infrastructure changes.
- Automatic deployment or rollback operations.
- Direct modification of production databases.
- General-purpose autonomous coding agents.
- Audio and video processing.
- Broad enterprise knowledge management across unrelated departments.

The initial product must remain focused on engineering knowledge and technical investigation.

13. Trust, Accuracy, and Safety Requirements

Trust is the highest product priority.

The system must follow these rules:

1. Important factual findings should be supported by source evidence.
2. Citations must point to the actual supporting document or passage.
3. The system must not fabricate document references.
4. Missing information must be communicated clearly.
5. Conflicting information must not be silently merged.
6. Reported causes must be distinguished from inferred hypotheses.
7. Recommendations must be distinguishable from documented facts.
8. Retrieved documents must be treated as untrusted content.
9. AI agents must only execute explicitly permitted tools.
10. All important execution failures must be observable.

The system should prefer an honest "insufficient evidence" response over a confident but unsupported answer.

14. Success Metrics

The project should demonstrate measurable results in the following areas.

Knowledge Retrieval

- Relevant passages are retrieved for evaluation questions.
- Retrieval results are traceable to source documents.

Answer Quality

- Answers are factually supported by retrieved evidence.
- Unsupported questions are handled appropriately.
- Conflicting sources are communicated.

System Reliability

- Document processing failures are visible.
- Provider failures are handled.
- Agent execution failures are recoverable.

Performance and Cost

- Response latency is measured.
- Token consumption is tracked.
- Provider usage is observable.

Product Usability

- Users can upload and manage documents.
- Users can ask questions and inspect citations.
- Users can continue conversations.
- Users can understand agent execution steps.

Specific numerical performance targets should be established after a baseline is measured using the evaluation dataset.

15. Important Architectural Decisions Pending

The following decisions must be finalized before database schema design and implementation.

Decision 1: Document Access Model

How should documents be shared?

Options:

A. Private Documents — Each engineer can access only their own uploaded documents.

B. Organization-Wide Documents — Authorized members of an organization can access shared documents.

C. Hybrid Model — Documents can be private or shared with an organization or team.

Status: Pending decision.

Decision 2: Knowledge Base Organization

How should technical knowledge be organized?

Options:

A. One Shared Knowledge Base — All authorized engineering documents belong to one common knowledge base.

B. Separate Team or Project Knowledge Bases — Documents are separated by engineering team or project.

C. Hybrid Knowledge Base — Organization-wide knowledge can coexist with separate team or project knowledge bases.

Status: Pending decision.

These choices will influence database relationships, document permissions, retrieval filters, and the overall architecture. They must not be silently assumed during implementation.

16. Assumptions and Constraints

The following assumptions guide the initial scope:

1. The primary users are engineering and technical operations professionals.
2. The initial knowledge domain is technical documentation.
3. PDF, TXT, Markdown, and CSV are the required input formats.
4. OpenAI and Anthropic are the initial LLM providers.
5. The platform will use RAG for evidence-based knowledge retrieval.
6. The platform will include a controlled AI agent with at least three tools.
7. The platform will maintain persistent conversation history.
8. The evaluation dataset will contain at least 20 questions.
9. The system must be deployed to a cloud environment.
10. The project must be demonstrable and explainable during supervised technical verification.

The initial implementation will prioritize working functionality, evidence quality, security, and clear architecture over unnecessary feature expansion.

17. Final Scope Statement

The Engineering Intelligence Platform is an AI-powered technical knowledge and investigation system for engineering teams.

It will allow authorized users to upload and organize technical documents, retrieve relevant knowledge through natural-language queries, receive evidence-backed answers, investigate technical incidents across multiple documents, interact with a controlled multi-tool AI agent, and maintain contextual conversations.

The platform will integrate multiple LLM providers, implement retrieval improvements, validate AI-generated responses, measure system quality through a repeatable evaluation framework, and expose its capabilities through a deployed web application.

The initial version will focus exclusively on engineering and technical operations knowledge.

Its defining product characteristic will be **trustworthy engineering intelligence through evidence, traceability, and controlled AI execution.**

Document End

Next Development Phase: Step 2 — System Architecture Design.

Prerequisite: Finalize the document access model and knowledge base organization decisions before beginning database schema design.